

From Latin Digits to Babylonian Cuneiform: Measuring the Provenance of Number Helices in LLMs

Girish Gupta*

Abstract

Kantamneni & Tegmark (2025) (K&T) show that LLMs encode integers $a \in [0, 99]$ as a *generalised helix*: a linear “number-line” axis plus modular circles at $T \in \{2, 5, 10, 100\}$. A natural reading is that the transformer constructs this geometry through depth. We ask where the geometry *comes from*, and report four measurements per cell: (i) the provenance ratio $\rho = R_{\text{helix}}^2(L=0)/\max_L R_{\text{helix}}^2(L)$, (ii) a basis-bandwidth and window-wrap check, (iii) a random-embedding control, and (iv) CKA between the $L=0$ and peak-layer helix subspaces. Across 8 models $\times$ 12 numeral scripts (96 cells), K&T’s central Latin result holds cleanly on their three models (Pythia-6.9B, Llama-3.1-8B, GPT-J-6B; $\rho = 0.34, 0.39, 0.41$) but that the apparent cross-script generalisation of the helix is largely *pre-transformer*: only 6–8 of 96 cells clear a joint (ρ , quality) bar for depth-built geometry. The clearest single result is on Babylonian cuneiform: replacing every learned embedding with a random vector at matched scale leaves the $L=0$ helix score unchanged on all 8 models ($|\Delta R^2| \leq 0.008$ in the native protocol). The measured base-60 geometry arises mechanically from how the renderer encodes wedges and how mean-pooling combines them, with no learned semantics required. Probes, SAEs, and any “the model represents X as a manifold” claim need a provenance check before attributing measured geometry to learning.

*girish@girishgupta.com

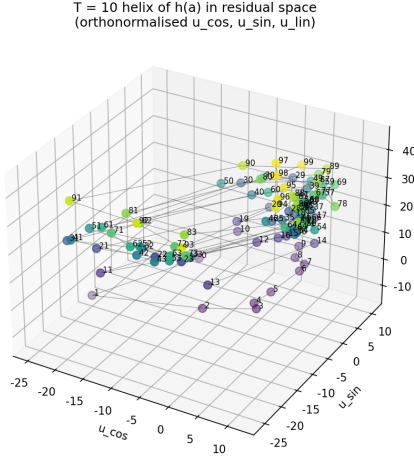


Figure 1: The number helix at $T = 10$ in Pythia-6.9B’s residual stream, Latin digits 0–99, peak layer. Two axes are the $(\cos 2\pi a/10, \sin 2\pi a/10)$ pair; the vertical axis is the linear “number-line” direction. Colour indicates a . Integers sharing the units digit land at the same angular position; magnitude lifts them along the vertical.

1. Introduction

Kantamneni & Tegmark (2025) reverse-engineer a striking representation: in the residual stream of three mid-sized LLMs (Pythia-6.9B, GPT-J-6B, Llama-3.1-8B), each integer $a \in [0, 99]$ is encoded as a generalised helix—a linear magnitude axis plus orthogonal modular circles at periods $T \in \{2, 5, 10, 100\}$ (Fig. 1 shows the $T = 10$ slice). The basis is itself the LLM-scale echo of Nanda et al. (2023), who reverse-engineered the trig solution in a tiny transformer trained on modular addition (itself an interpretation of the Power et al. (2022) grokking dynamics). K&T further show the helix is causally used for addition.

K&T deliberately avoid many tokenization complications by focusing on space-prefixed Latin integers that are single tokens in their models—GPT-J tokenizes $[0, 361]$ as one token, Pythia $[0, 557]$, Llama $[0, 999]$ —which is the right clean setting for their causal addition study. Our question starts where that simplification stops. When the same helical fit is generalised across scripts and bases, the input pipeline itself can create or preserve Fourier-fittable structure, so provenance becomes a non-trivial question. A high helix R^2 does *not* tell us where the geometry came from. It might be constructed by transformer depth, or inherited from the input pipeline (rendering, tokenization, embedding lookup, pooling), or induced by the measurement protocol itself (wrong basis, too-short input range). This question matters: in the literature, “the model represents X as a [helix/sphere/manifold]” is routinely read as a claim about what the transformer does, when it can in principle be a claim about how the inputs were prepared.

This provenance question is also motivated by behavioral work on number tokenization. Singh & Strouse

(2024) show that arithmetic performance in frontier LLMs depends strongly on how numbers are tokenized, with systematic error patterns at token boundaries. Their result is behavioral; ours is representational. If tokenization can change arithmetic behavior, can the input pipeline also *create* the geometry we later interpret as a learned representation?

Contributions. We describe four measurements (§2) and what they say across 8 models $\times$ 12 numeral scripts (§3). We report three findings (§4). (1) K&T’s Pythia/Llama/GPT-J Latin cells survive cleanly under the wider protocol; the depth-built finding holds where the original paper claimed it. (2) The “helix-in-script- X -too” generalisation is mostly pre-transformer: only 6–8 of 96 cells qualify as depth-built or depth-amplified under a joint ρ +quality criterion. (3) Babylonian cuneiform is the cleanest case of mechanically-produced geometry: swapping every learned embedding for a random vector leaves the $L=0$ helix score unchanged on all 8 models. We discuss implications for probing and SAE interpretability (§5).

Complementary causal work: Feucht, Haklay et al. (2026) identify a layer-18 base-10 addition module in Llama-3.1-8B that reads from exactly the Fourier features the helix basis fits, on exactly the cell our measurements mark as depth-built—answering the *what is the geometry used for?* question on a cell we mark as the geometry-was-actually-built one.

2. What We Measure Per Cell

A high helix R^2 at the best layer is a measurement; on its own it doesn’t tell us how the geometry got there. To answer the provenance question we report four separate measurements per (model, script) cell: (i) the helix score and how dominant the helix subspace is among all K -d fits; (ii) where in the stack the helix actually lives, and how much of it is already at the embedding output; (iii) whether the basis and window can in principle see the script’s natural period; and (iv) when depth does change the score, whether it preserves the geometry or replaces it. Let $H \in \mathbb{R}^{n \times d}$ be the residual stream at layer L over n integers, with $h(a)$ the row for integer a .

Helix basis and quality. The trig basis is

$$B(a) = [a, \cos(2\pi a/T), \sin(2\pi a/T)]_{T \in \mathcal{T}}, \quad (1)$$

giving $1 + 2|\mathcal{T}|$ features per integer (9 features at $\mathcal{T} = \{2, 5, 10, 100\}$). Each column has a geometric meaning. The linear column a is the “spine”; when OLS fits it, the column picks out the residual-stream direction that best correlates with integer magnitude—the number-line direction. Each $(\cos 2\pi a/T, \sin 2\pi a/T)$ pair traces the unit circle as a varies, completing one revolution per T integers, so integers sharing the same $(a \bmod T)$ land at the same angular position. $T = 2$ separates parity, $T = 5$

Table 1: The twelve numeral scripts, grouped by family. “Native T ” is the period a basis would have to include to fit the script’s positional structure exactly. The paper-default basis catches Latin and the other positional-base-10 scripts; binary/hex/Babylonian motivate extended-basis runs (App. C).

script	23 renders as	24 renders as	native T	family
Latin	23	24	10	positional base-10
Arabic-Indic	٢٣	٢٤	10	positional base-10
Persian	۲۳	۲۴	10	positional base-10
Devanagari	२३	२४	10	positional base-10
Thai	๒๓	๒๔	10	positional base-10
CJK positional	二三	二四	10	positional base-10
Binary	10111	11000	2–64	positional base-2
Hexadecimal	17	18	16–256	positional base-16
Greek alph.	κγ	κδ	—	additive (Milesian)
Hebrew alph.	כג	כד	—	additive (gematria)
Roman	XXIII	XXIV	—	additive
Babylonian	<<	<<	60	positional base-60, additive cols

gives five clusters by mod-5, $T = 10$ gives the canonical clock face for the units digit, $T = 100$ gives coarse position over the whole 0–99 range. Stacked, the result is a helix: the cos/sin pair rotates as a increases and the linear column lifts each point. We keep cos and sin separated (rather than using the complex exponential $e^{2\pi ia/T}$) because the fit is real-valued OLS and because each (cos, sin) pair gives the model a 2D plane it can *rotate in*—which is exactly the “Clock” algorithm K&T identify, where addition becomes rotation by the addend’s angle on each clock.

We fit $H \approx BW$ by ordinary least squares and report the variance-weighted R^2 at every layer. The variance weighting puts emphasis on the hidden dimensions that actually vary in a ; otherwise the score would be dominated by the many residual-stream dimensions that are nearly constant across integers (tokens for “language”, “decimal”, etc.). The columns of W tell us *where* each clock and the number-line direction live in residual space—and Figure 1 is constructed by extracting the three columns (cos $2\pi a/10$, sin $2\pi a/10$, a), QR-orthonormalising them into a clean 3D frame, and projecting H onto it.

A high helix R^2 alone is not enough: any sufficiently expressive basis fits a low-dimensional signal. The relevant comparison is the K-d PCA upper bound at the same dimensionality (the Eckart–Young optimal K -feature reconstruction). The ratio $R^2_{\text{helix}}/R^2_{\text{PCA}}$ is our *quality* metric: how dominant the helix subspace is among all K -d fits. A ratio near 1 means the model isn’t just *fitting* the helix, it’s *using* it; a ratio near 0 means the basis fits little that the activations contain.

Provenance ratio ρ . The core measurement:

$$\rho = \frac{R^2_{\text{helix}}(L=0)}{\max_L R^2_{\text{helix}}(L)}. \quad (2)$$

The numerator and denominator answer different questions about the same cell. The denominator is the helix

score K&T’s analysis already reports: how well the trig basis fits at the model’s most helix-friendly layer. The numerator scores the same basis at $L = 0$, the residual stream *before any transformer block has run*—the output of token embedding, positional embedding, and mean-pooling over the numeral’s sub-tokens. Convention note: our $L = 0$ is HuggingFace `hidden_states[0]`, the pre-block embedding+pooling output. This is one step *before* K&T’s “following layer 0” residual stream, which already has one block of computation applied; the pre-block measurement is what we want for the provenance question because it isolates the input pipeline. So ρ asks: “of the helix you measure at the peak layer, how much was already supplied by the input pipeline before the transformer ran?” $\rho = 1$ means all of it (inherited); $\rho \rightarrow 0$ means depth genuinely builds the geometry (depth-built); intermediate ρ means depth amplifies a real but partial $L=0$ head start.

Two notes on what ρ is not. First, ρ alone is insufficient: in a cell where the basis fits little anywhere, a low ρ just means “ratio of two small numbers,” not “hidden depth-built helix.” We therefore require a quality threshold alongside ρ . We call a cell *depth-built* when $\rho \leq 0.55$ and $R^2_{\text{helix}}/R^2_{\text{PCA}} \geq 0.70$; *depth-amplified* when $0.55 < \rho \leq 0.70$ at quality ≥ 0.60 ; *inherited* when $\rho \geq 0.80$ at quality ≥ 0.55 ; *weak* when quality < 0.55 ; *ambiguous* between thresholds. Second, ρ is silent about *what* depth does when it does work—two cells can share the same ρ while depth treats their $L=0$ geometry very differently. The CKA check below recovers that distinction.

Basis bandwidth and window wrap. The trig basis is not universal; it can only fit periods it explicitly includes. A model might encode integers using a period T the basis doesn’t contain, and the score will look low through no fault of the model. Two failure modes follow.

Mode 1 (basis bandwidth): if the script’s natural pe-

riod isn't in $\mathcal{T}$, the basis is blind to it. This is analogous to trying to detect a 1 kHz tone with a microphone that only captures 5–20 kHz: the tone is there, the microphone reports “no signal,” and that is a fact about the microphone, not the room.

Mode 2 (window wrap): even if we put $T = 60$ in the basis, $n_{\max} = 100$ wraps $T = 60$ only once (around $a = 60$). One wrap is not enough to identify a periodic component. The Fourier intuition: the FFT of a length-100 signal at frequency $f = 1/60$ doesn't form a clean single bin because 100 isn't an integer multiple of 60. The energy of a true $T = 60$ component spreads across multiple frequency bins (“spectral leakage”), making it indistinguishable from noise. With ten cycles in 600 samples, leakage is small and the peak at $1/60$ is sharp and unambiguous. The regression intuition is the same in different language: with one wrap of $T = 60$ over 100 samples, the columns $\cos(2\pi a/60)$ and $\sin(2\pi a/60)$ are highly correlated with all sorts of other columns and with each other; OLS can fit “something” but the result is unstable and not attributable to $T = 60$ structure specifically. Rule of thumb: you need the input range to wrap any candidate period at least 8–10 times for clean identification.

Both modes hide real structure. Babylonian cuneiform (native $T = 60$), binary (natural periods $T \in \{2, 4, 8, 16, 32, 64\}$), and hex (natural periods $T \in \{16, 32, 64, 256\}$) all fail at least one bar at paper defaults. The mitigation is conservative: widen $n_{\max}$ past several wraps of any candidate period, and explicitly add the script's native period to $\mathcal{T}$. If the score jumps with the fix, the original measurement was hiding real structure; if it doesn't, the script genuinely has no helix.

Random-embedding control. The most direct test of whether the $L=0$ helix is mechanical. Replace the model's learned embedding matrix with $\tilde{E} \sim \mathcal{N}(0, 1/\sqrt{d})$ at matched per-coordinate scale, recompute the $L=0$ representation (the tokenizer, the renderer, and mean-pooling are unchanged; only the embedding lookup is replaced), and re-fit the helix. The gap $\Delta R_{L=0}^2 = R_{\text{learned}}^2 - R_{\text{random}}^2$ is what learned embeddings contribute on top of the rendering+pooling floor. A small Δ means the $L=0$ geometry is mechanical: the embedding table contributed nothing the input pipeline didn't already produce. A large Δ means the embedding table carries learned structure beyond what the rendering+pooling mechanically generates. This is the strongest version of the input-pipeline check, and the one that gives the paper's cleanest single number (§4.3).

Representation alignment (CKA). The first three measurements characterise the *score* a cell achieves and where that score lives. They are silent about *shape*. Two cells can share the same high ρ —both with most of the helix already at $L=0$ —while one of them lets the

$L=0$ geometry pass through unchanged and the other replaces it with a substantially different integer-to-integer pattern that just happens to fit the trig basis equally well. Linear CKA (Kornblith et al., 2019) between the helix-projected $L=0$ and peak-layer representations distinguishes the two cases. Letting $X = BW^{(L=0)}$ and $Y = BW^{(\text{peak})}$, $\text{CKA}(X, Y) \in [0, 1]$ measures whether the *integer-to-integer distance pattern* is the same at the two depths. $\text{CKA} \approx 1$ at high ρ means *pass-through*: the input pipeline supplies the geometry and depth doesn't touch it. $\text{CKA} \ll 1$ at high ρ means *rebuild*: depth replaces the $L=0$ helix with a similar-score but differently-shaped one. CKA is invariant to rotation and rescaling in feature space, so “different geometry” here means the *row-similarity pattern* differs, not necessarily that the hidden directions are orthogonal.

3. Experimental Setup

Models (all base, non-instruction-tuned): Pythia-6.9B, GPT-J-6B, Llama-3.1-8B, Gemma-4-E4B, Gemma-4-31B, OLMo-3-32B, Qwen2.5-7B, Qwen2.5-32B—a superset of the K&T model set (Pythia, GPT-J, Llama) chosen to cover three architectural axes the original paper does not: a multilingual family with strong Indic/Arabic-Indic exposure (Gemma 4), a model family that aggressively splits numbers into per-digit tokens (Qwen 2.5), and a recent base model with a well-documented training mix (OLMo-3). Sizes range from 4 B (Gemma-4-E4B) to 32 B (Qwen2.5-32B, OLMo-3-32B, Gemma-4-31B). Layer counts range from 28 (Pythia, GPT-J) to 64 (OLMo, Qwen2.5-32B); the helix's peak layer varies enormously across this set (App. B).

Scripts (12 total, four families); Table 1 gives each script's rendering of 23 and its mathematical family. **Pooling.** K&T focus on the *single-token regime* “for simplicity”—GPT-J tokenizes integers in $[0, 361]$ as one token, Pythia $[0, 557]$, Llama $[0, 999]$ —so for Pythia/GPT-J/Llama on Latin at $n_{\max} = 100$, every integer is one token and the choice of pool is moot. Outside that regime the choice matters: Qwen and Gemma split Latin numbers per-digit even at $n = 100$, every non-Latin script is multi-token on every tokenizer, and our extended-basis Babylonian/binary/hex runs operate well outside any model's single-token range. We use mean pooling throughout, which is the honest extension—last-token pooling produces a period-10 artifact because numbers 23, 33, 43, ... all share the same final sub-token in per-digit BPE; mean pooling is also the channel through which the rendering-only structure on Babylonian (§4.3) becomes visible. The default protocol is $n_{\max} = 100$, $\mathcal{T} = \{2, 5, 10, 100\}$; Babylonian/binary/hex additionally run with wider windows and native bases (App. C).

For each (model, script) cell we run one forward pass per integer with `output_hidden_states`, mean-pool the numeral's sub-token positions at every layer, fit the helix basis, and record per-layer R^2 and ρ . Inputs are space-

prefixed (“␣23” rather than bare “23”), matching K&T’s convention—GPT-style tokenizers can split “23” and “23” into different tokens, and natural prompts almost always have a leading space or operator. Code, configurations, and all 8 GB of intermediate JSON are public; see App. G.

4. Results

4.1. The paper-models / Latin cells survive cleanly

The first check is whether our measurements reproduce the original paper’s finding on the cells where the original protocol is the right one. On K&T’s three models reading Latin digits, ρ is well below the inherited threshold and quality is high: Pythia $\rho = 0.34$ ($R^2_{\text{helix}}/R^2_{\text{PCA}} = 0.85$), Llama $\rho = 0.39$ (0.85), GPT-J $\rho = 0.41$ (0.83). Less than half of the helix is at $L=0$; the rest is genuinely built by depth. All three cells clear the depth-built bar comfortably, and the random-embedding control on Latin (App. D) shows the $L=0$ helix on these models is already substantially learned: random embeddings of matched scale recover only $R^2 \approx 0.08$ at $L=0$ (essentially chance) compared to the learned 0.18–0.26, leaving $\Delta R^2 = +0.10$ to $+0.15$ for the embedding contribution alone. The K&T result is intact on the cells where they claimed it.

Where the helix lives in the stack. K&T originally read at the middle layer ($n_{\text{layers}}/2$). For Pythia (32 layers), that is $L=16$; for Llama (32 layers), $L=16$. Both have peaks near there, so the middle-layer heuristic was fine for K&T’s three models. It is the wrong heuristic for any of the five models we add. Pythia’s actual peak is at $L=32$ of 32 (the literal final layer); GPT-J’s at $L=28$ of 28; Llama’s at $L=15$ of 32 (close to the middle); Qwen2.5-7B/Latin peaks at $L=0$, before any block has run (the helix is complete in the embedding+pooling output); Qwen2.5-32B/Latin peaks at $L=3$ of 64; Gemma-4-31B holds a moderate helix R^2 across most of the stack with a sharp spike to peak at $L=29$ of 60; Gemma-4-E4B has a double-peaked structure with global maximum at $L=17$ of 42; OLMo-3-32B rises gradually to peak at $L=23$ of 64. Without the layer sweep we could not compute ρ at all (there would be no $\max_L R^2$ to ratio $L=0$ against), and the sweep is also what lets us target our standard figures (FFT, PC1, circles, 3D helix) at each model’s actual peak rather than at K&T’s middle-layer default. Per-model curves are in App. B.

4.2. Cross-script generalisation is mostly pre-transformer

A natural next step is to ask whether the helix story generalises beyond the cells K&T tested. The raw helix matrix *looks* like it does: many non-Latin cells fit the trig basis non-trivially. The provenance check shows that

most of that apparent generalisation is pre-transformer.

Figure 2 shows ρ for all 96 cells at the paper-default protocol. The cool (depth-built) row is visually narrow: outside the three paper-model Latin cells, depth substantially contributes only on Gemma’s strong non-Latin cells (Devanagari, Persian, Roman; $\rho = 0.44$ – 0.63)—scripts where Gemma 4 plausibly had heavy training exposure, though we do not measure pre-training distributions directly. Everything else outside Latin is warm. Most non-Latin cells on Pythia and Llama sit at $\rho = 0.73$ – 1.00 : the helix is already present at $L = 0$ because the tokenizer splits e.g. ४३ into per-digit sub-tokens whose mean (over the two positions) varies smoothly with the digit values. The trig basis fits that smooth variation as a helix without any depth being involved.

Two models take pre-transformer geometry to its limit even on Latin. Qwen2.5-7B/Latin has $\rho = 1.00$ exactly: the helix peaks at the literal embedding output before any block has run. Qwen2.5-32B/Latin peaks at $L=3$ of 64 ($\rho = 0.89$). On Qwen 2.5 the Latin cell that K&T’s reading would call depth-built on Pythia/Llama is instead near-pure inheritance—a function of how Qwen’s tokenizer breaks Latin numbers and how mean-pooling combines the sub-tokens.

Greek alphabetic numerals are uniformly weak ($\rho \geq 0.94$ on every model that does not actively train on Greek): Greek 23 is $\kappa\gamma$, two letters from disjoint alphabet blocks with no positional period-10 structure. There is no helix to inherit and no helix for depth to build. The mutually-exclusive classifier of §2 marks 6 of 96 cells as depth-built and 4 as depth-amplified at paper defaults—about 10% of the matrix. The helix framing borrows rhetorical strength from the three paper-model Latin cells to characterise cells where the actual mechanism is quite different.

4.3. Babylonian: geometry from rendering alone

Babylonian cuneiform is positional at base 60 (additive within each sexagesimal column) and so fails both spectral checks at the paper defaults: $T = 60$ is not in $\mathcal{T} = \{2, 5, 10, 100\}$, and a window of $n_{\text{max}} = 100$ wraps $T = 60$ only once. The natural protocol fix—widen n_{max} to 600 and add $T = 60$ to $\mathcal{T}$ —recovers 8–16 percentage points of helix R^2 uniformly across all 8 models, with the largest gain on Qwen2.5-7B (+0.16). The basis-free FFT spectrum at the wider window confirms peaks at multiples of $1/60$. A reading focused on the score would stop here: cuneiform on every model carries period-60 structure once you measure properly.

The random-embedding control reframes that conclusion sharply. With the native protocol fixed, we run the control of §2: replace the model’s embedding table with a matched-scale Gaussian random matrix and recompute the $L=0$ helix score. Table 2 reports the learned-vs-random gap. The maximum absolute $|\Delta|$ across all eight models is 0.0073; two models have neg-

	lat	arab	pers	deva	thai	cjk	bin*	hex*	grk	heb	rom	bab*
Pythia-6.9B	.34	1.00	.99	.99	.98	.95	.51	.64	1.00	.95	.78	.86
GPT-J-6B	.41	1.00	.99	1.00	1.00	1.00	.51	.66	1.00	.96	.76	.85
Llama-3.1-8B	.39	1.00	.73	1.00	1.00	.99	.44	.77	1.00	1.00	.73	.87
Gemma-4-E4B	.89	.80	.63	.49	1.00	.96	.59	.43	.56	.80	.55	.89
Gemma-4-31B	.79	.74	.61	.44	.81	.88	.50	.34	.55	.75	.48	.88
OLMo-3-32B	.63	.92	.98	.85	1.00	1.00	.54	.77	1.00	1.00	.65	.82
Qwen2.5-7B	1.00	.89	.87	.77	.92	.91	.78	.73	.97	.90	.70	.93
Qwen2.5-32B	.89	.93	.91	.76	.88	.89	.64	.69	.94	.85	.67	.92

Figure 2: ρ at paper defaults ($n_{\max} = 100$, $\mathcal{T} = \{2, 5, 10, 100\}$). Teal = depth-built ($\rho \leq 0.45$); light teal = depth-amplified (0.45–0.60); cream = ambiguous; orange = inherited (0.80–0.95); deep orange = $\rho > 0.95$. Starred columns are mismeasured under paper defaults; see Table 2 and App. C for native bases.

ative Δ (the random fit is marginally *better* than the learned one). The L=0 Babylonian helix is not learned: it is mechanical.

The mechanism is direct. Walking through one concrete integer makes it explicit. The pipeline that produces $h(n)$ at $L = 0$ is the same four steps for every cell: *rendering* $\rightarrow$ *tokenization* $\rightarrow$ *embedding lookup* $\rightarrow$ *mean-pooling*. The random-embedding control swaps the third step only. For $n = 23$ on Babylonian:

Step 1 (rendering). `format_number(23, ``babylonian'')` returns `<<|||`: two ten-wedges followed by three one-wedges. The structure is additive—the number of `<` equals the tens digit, the number of `!` equals the ones digit.

Step 2 (tokenization). The space-prefixed string is fed to the tokenizer. Cuneiform was not in any of our eight models’ pre-training corpora to speak of, so each codepoint falls back to its UTF-8 byte representation (four bytes per cuneiform codepoint, often each becoming its own sub-token). Crucially, every `<` expands to the same fixed byte-token sequence, and likewise for `!`. Writing T for the byte-tokens of one `<`, the tokenisation of 23 is `[space, T, T, T, T, T]`. The count structure of step 1 is preserved.

Step 3 (embedding lookup). Each token ID indexes a row of the embedding matrix $E \in \mathbb{R}^{V \times d}$. Write e for the contribution of one T (the sum of the byte-token rows). The result is the sequence `[espace, e, e, e, e, e]`.

Step 4 (mean-pooling). Average:

$$h(23) \approx \frac{e_{\text{space}} + 2e + 3e}{6}. \quad (3)$$

This is a linear combination of three *fixed* vectors with coefficients that come from the renderer’s count structure: 2 = the tens digit of 23, 3 = the ones digit. Generalising for $n < 60$,

$$h(n) \approx \alpha(n)e + \beta(n)e + \text{constant}, \quad (4)$$

where $\alpha(n) = \lfloor n/10 \rfloor$ and $\beta(n) = n \bmod 10$. Both are smooth/periodic functions of n ; the trig basis with $T \in \{10, 60\}$ in $\mathcal{T}$ fits the coefficient structure.

What the random-embedding control changes. Steps 1, 2, 4 are unchanged. Only step 3 changes: e and e are replaced by independent draws from $\mathcal{N}(0, 1/\sqrt{d})$. So

$$\tilde{h}(n) \approx \alpha(n)\tilde{e} + \beta(n)\tilde{e} + \text{constant}. \quad (5)$$

Table 2: Babylonian random-embedding control at the native protocol ($n_{\max} = 600$, $\mathcal{T} = \{2, 5, 10, 60, 100\}$). All eight $|\Delta| \leq 0.008$. The L=0 helix score replicates with random vectors of matched scale.

model	learned $R^2_{L=0}$	random $R^2_{L=0}$	Δ
Pythia-6.9B	0.569	0.568	+0.001
GPT-J-6B	0.566	0.567	−0.002
Llama-3.1-8B	0.517	0.513	+0.003
Gemma-4-E4B	0.623	0.620	+0.002
Gemma-4-31B	0.621	0.621	−0.001
OLMo-3-32B	0.545	0.552	−0.007
Qwen2.5-7B	0.588	0.590	−0.002
Qwen2.5-32B	0.587	0.592	−0.004

The vectors $\tilde{e}$ and $\tilde{e}$ now point in random directions in $\mathbb{R}^d$, but they are still *fixed* vectors, and the coefficients $\alpha(n)$ and $\beta(n)$ are the same smooth functions of n . The trig basis fits the coefficient structure regardless of which directions the random vectors point. The fitted helix *rotates* in residual space with each random seed; the fit *quality* doesn’t change. This is the algebra behind Table 2.

The contrast with Latin/Pythia is sharp. Pythia tokenizes “23” as a single token, so the chain collapses to $h(23) = e_{23}$. The helix R^2 is asking whether the 100 learned embeddings $\{e_0, e_1, \dots, e_{99}\}$ themselves lie on a helix. With learned Pythia weights they do; with random embeddings they don’t, because 100 independent random vectors carry no inter-integer structure. So the same swap-embeddings control gives sharply different verdicts on the two cells—and the difference is fully explained by which step in the four-step chain carries the geometry.

This is the strongest example of the warning Singh & Strouse’s behavioral result implies at the representational level: tokenization is not a neutral preprocessing step but an inductive bias in the numerical reasoning pipeline. On Babylonian, the bias is so strong that it generates the entire L=0 helix score on its own.

The contrast with Latin is sharp. On Latin every model has a substantial positive Δ (+0.03 to +0.18; full table in App. D). On Pythia/Llama/OLMo/GPT-J the random-embedding R^2 at $L = 0$ is around 0.08—chance, since the Latin tokenizer gives those models a single to-

ken per integer and random vectors carry no number information. The learned L=0 helix on Latin (0.18–0.26 on the same four models) reflects learned digit semantics. Two cells, two different mechanisms: Latin’s L=0 helix on Pythia is learned digit geometry that depth then refines; Babylonian’s L=0 helix on the same model is rendering+pooling arithmetic that has nothing to do with what the embedding table contains.

4.4. Pass-through versus rebuild

ρ alone conflates two mechanistically distinct high- ρ regimes. The CKA check from §2 separates them. A concrete comparison makes the distinction sharp.

Two cells with similar ρ . Qwen2.5-32B/Latin and Gemma-4-31B/Latin both have high ρ (0.89 and 0.79): on both cells, the helix score at the peak layer is mostly already at $L = 0$. A ρ -only reading puts them in the same bucket: “inherited; the transformer doesn’t do much.” Now look at CKA on the helix-projected representations: Qwen-32B/Latin has CKA = 0.91, Gemma-31B/Latin has CKA = 0.33. Same ρ , very different CKA.

What that means geometrically. Imagine the 3D helix at $L = 0$ with integers labelled, then the 3D helix at the peak layer (same model, same script). For Qwen-32B, the two plots are nearly identical: integer 23 is in the same position on the helix in both, integer 47 is in the same position, and so on. The transformer left the arrangement alone—*pass-through*. For Gemma-31B, the two plots have the same overall helical shape, but the labels have moved: integer 23 might be at the back of the L=0 helix and at the front of the peak helix. The score is the same (both fit the trig basis equally well), but it’s a *different* helix—the row-similarity pattern across integers has changed. This is *rebuild*.

Linear CKA captures exactly this distinction. It is invariant to rotation and rescaling of the feature columns, so it isn’t asking “are the two helices in the same place in residual space?”—they obviously aren’t, depth has done work. It’s asking “is the row-similarity pattern the same?”—i.e., are the same integers close to each other in both helices? CKA ≈ 1 on Qwen-32B says yes; CKA ≈ 0.33 on Gemma-31B says no.

Three patterns across the matrix. *Pass-through* (high ρ , high CKA): Qwen2.5-32B/Latin (0.89, 0.91), OLMo/Latin (0.63, 0.91), most Pythia and Llama non-Latin positional cells (CKA ≥ 0.98 on Persian, Devanagari, Chinese on Pythia). The input pipeline does all the work and the transformer passes the geometry through. *Rebuild* (high ρ , low CKA): the entire Gemma family (CKA = 0.21–0.65 across most scripts), and a third pattern that recovers Babylonian’s full story. On Pythia/Llama/OLMo, Babylonian has $\rho \approx 0.86$ but CKA = 0.25–0.53: the peak Babylonian helix is depth-built *on top of* a mechanical L=0 starting point. The score doesn’t change much (the mechanical L=0 is already healthy), but depth rebuilds the integer-to-integer pattern. What that rebuilt

helix is computing is a separate, causal question—one ρ + CKA flags but doesn’t answer.

The full (ρ , CKA) scatter and a four-quadrant breakdown are in App. E.

4.5. Binary and hex: two more case studies in basis bandwidth

Babylonian is not the only script where the paper basis is the wrong basis. Binary’s natural periods are powers of two ($T \in \{2, 4, 8, 16, 32, 64\}$), of which $T = \{2, 5, 10, 100\}$ catches only $T = 2$; hexadecimal’s natural periods are $T \in \{16, 32, 64, 256\}$, none of which paper basis catches. Running both at $n_{\max} = 1024$ with the native basis recovers +0.10 to +0.37 helix R^2 on binary and +0.19 to +0.29 on hex (App. C). The protocol fix is the same; the mechanistic readings on the two scripts are entirely different.

Binary is mechanical, like Babylonian. The random-embedding gap with the binary-native basis is $|\Delta R^2| \leq 0.03$ on every model: the L=0 helix on binary is generated by the renderer (binary strings have length $\log_2 n$, whose count varies with n) and the tokenizer (BPE chunks of bits) interacting with mean-pooling. Pythia/Llama/GPT-J/OLMo carry moderately-low native-basis ρ of 0.62–0.77 on binary, but this reflects depth doing additional work on top of a mechanical L=0, not learned bit semantics; the corresponding CKA values (0.55–0.71) confirm depth amplifies rather than rebuilds. Binary is the second mechanical case in the matrix.

Hex is depth-built, the second-cleanest depth-built case in the matrix. Native-basis ρ on Pythia/Llama/GPT-J/OLMo is 0.29, 0.30, 0.35, 0.35—squarely in the same regime as the original paper’s Latin cells ($\rho = 0.34, 0.39, 0.41$). The random-embedding gap is positive on every model (small on Llama/OLMo, sizeable on Gemma), so hex digit embeddings do carry learned structure that random vectors don’t reproduce. The natural hypothesis is that all these models saw substantial hex during pre-training (code, addresses, hashes), but the data here doesn’t directly test pre-training distributions. Whatever the cause, on the four models with the cleanest result, the model is treating hex digits the way it treats Latin digits: as indices into a value space, with layer-by-layer construction of the same kind of helix.

The general pattern across the three case studies: each non-decimal base needs its own protocol fix before its ρ becomes interpretable, and once the fix is applied, the rest of the measurements cleanly separate “mechanical inheritance” (Babylonian, binary) from “depth-built” (hex on the paper models) within the same broad failure mode.

5. Discussion

The reframing is the contribution, not the matrix. Three problems can make the standard “fit a helix, report R^2 ” procedure answer the wrong question. Two of them hide

real structure: the basis can miss the period the model actually uses (§4.3, §4.5), and the window can be too short to identify a real period (Babylonian at $n_{\max} = 100$). The third credits the transformer with structure it did not build—the case where the helix score is high because rendering, tokenization, embedding lookup, and mean-pooling already produced the geometry. ρ catches this, and the random-embedding control isolates how much of the $L=0$ geometry is mechanical versus learned. CKA then reveals a fourth: a high ρ does not always mean the geometry is untouched. Sometimes depth rebuilds the integer-to-integer pattern with similar quality but different shape, and a ρ -only analysis would lump that case in with genuine pass-through.

Across the matrix, the cells where K&T’s reading is unambiguously correct are narrower than the generalisation suggests: 6 of 96 cells at paper defaults clear the joint $\rho +$ quality bar for depth-built or depth-amplified geometry, concentrated on the three paper-model Latin cells and Gemma’s strong non-Latin scripts (App. F). Outside that narrow set, the apparent cross-script generalisation of the helix is mostly pre-transformer.

The Babylonian random-embedding result is the sharpest version of a general warning: *some apparent neural geometry can come from rendering statistics alone*, before learned embeddings or transformer blocks are even needed. Smooth, low-dimensional, easy-to-fit manifolds can be artefacts of how the input is encoded. For number helices specifically, $\rho +$ CKA together with the two spectral checks are how we asked the provenance question. The general lesson is independent of the specific measurements: for other geometric claims—probe-discovered features, sparse-autoencoder directions, “the model represents X as a manifold”—the right baseline is not zero or noise; it is the geometry the input pipeline already produces with random embeddings.

The reframing does not invalidate the existing literature. K&T’s Latin results survive cleanly, and the replication of the helix across eight models on Latin shows it is an architecturally general object even if its provenance is not. Babylonian’s base-60 result is then a substantive empirical statement about input pipelines—an additive symbolic renderer composed with mean-pooling is enough to produce a measurable helix. Causal work like Feucht, Haklay et al. (2026), which finds a base-10 addition module in Llama-3.1-8B reading exactly the Fourier features the trig basis fits, lands on exactly the cell our measurements mark as depth-built: the natural follow-up is to apply that kind of causal interrogation to the cells where $\rho +$ CKA flag a high score with no depth-built helix underneath.

References

Kantamneni, S. and Tegmark, M. Language models use trigonometry to do addition. *arXiv preprint arXiv:2502.00873*, 2025.

Nanda, N., Chan, L., Lieberum, T., Smith, J., and Steinhart, J. Progress measures for grokking via mechanistic interpretability. In *ICLR*, 2023. arXiv:2301.05217.

Power, A., Burda, Y., Edwards, H., Babuschkin, I., and Misra, V. Grokking: Generalization beyond overfitting on small algorithmic datasets. *arXiv preprint arXiv:2201.02177*, 2022.

Singh, A. K. and Strouse, D. J. Tokenization counts: the impact of tokenization on arithmetic in frontier LLMs. *arXiv preprint arXiv:2402.14903*, 2024.

Feucht, S., Haklay, T., et al. Arithmetic in the wild: causal base-10 features in Llama 3. *arXiv preprint arXiv:2605.01148*, 2026.

Kornblith, S., Norouzi, M., Lee, H., and Hinton, G. Similarity of neural network representations revisited. In *ICML*, 2019.

A. Methods details

Activation collection. For each integer a , we feed “<bos> format_number(a , script)” through the model with `output_hidden_states=True` and pool the numeral’s sub-token positions at every layer. We use mean pooling throughout; last-token pooling produces a period-10 artefact for multi-token scripts because numbers 23, 33, 43, ... all share the same last sub-token in per-digit BPE.

Helix fit. For numbers $\{0, \dots, n_{\max} - 1\}$ and periods $\mathcal{T}$, we construct B of shape $(n_{\max}, 1 + 2|\mathcal{T}|)$ from Eq. 1, then solve $H \approx BW$ by ordinary least squares with intercept. Report variance-weighted R^2 . The Eckart–Young upper bound is the variance-weighted reconstruction R^2 of the best $(1 + 2|\mathcal{T}|)$ -d PCA of H ; the helix/PCA ratio is our quality metric.

ρ and the peak layer. We collect activations at every `hidden_states[L]` (one forward pass per a), so $L = 0$ is the embedding output (after the token + positional embeddings and pooling, before any transformer block). We compute the helix R^2 at every L and define ρ from Eq. 2.

Random-embedding control. For a model with embedding matrix $E \in \mathbb{R}^{V \times d}$ we draw $\tilde{E} \sim \mathcal{N}(0, 1/\sqrt{d})$ (matched per-coordinate scale), use the model’s actual tokenizer to obtain sub-token id sequences for each a , mean-pool $\tilde{E}$ rows over those sub-tokens to obtain $\tilde{h}(a)$, fit the helix on $\tilde{H}$, and compare $\tilde{R}^2$ to the learned $R^2_{L=0}$. The tokenizer + renderer + pooling are identical to the learned condition; only the embedding lookup is replaced. No transformer blocks are run.

Linear CKA. For mean-centred $X \in \mathbb{R}^{n \times d_x}$, $Y \in \mathbb{R}^{n \times d_y}$, $\text{CKA}(X, Y) = \|X^\top Y\|_F^2 / (\|X^\top X\|_F \cdot \|Y^\top Y\|_F)$. We report CKA on the helix-projected representations $X = BW^{(L=0)}$, $Y = BW^{(\text{peak})}$, isolating whether the *helix slice* of the residual stream is the same at the two depths, not whether the rest of the model state is.

Classification thresholds. Mutually exclusive criteria: *depth-built* requires $\rho \leq 0.55$ and helix/PCA ≥ 0.70 ; *depth-amplified* requires $0.55 < \rho \leq 0.70$ and helix/PCA ≥ 0.60 ; *inherited* requires $\rho \geq 0.80$ at helix/PCA ≥ 0.55 ; *weak* is helix/PCA < 0.55 ; *ambiguous* is everything else (e.g. $0.70 < \rho < 0.80$). The bar is deliberately strict; relaxing helix/PCA to 0.60 adds Gemma’s two Roman cells.

B. Per-model peak layers on Latin

Where the helix lives varies dramatically across models. Pythia ($n_L = 32$) rises monotonically and peaks at the literal final layer. GPT-J ($n_L = 28$) does the same. Llama-3.1-8B ($n_L = 32$) peaks mid-stack at $L=15$. Gemma-4-E4B ($n_L = 42$) shows two regimes with a broad peak at $L=17$. Gemma-4-31B ($n_L = 60$) holds ~ 0.5 across most of the stack with a sharp spike to 0.70 at $L=29$. OLMo-3-32B ($n_L = 64$) rises through mid-stack to a peak at $L=23$. Qwen2.5-7B/Latin peaks at the literal embedding output ($L=0$; the helix is complete *before any block has run*). Qwen2.5-32B peaks at $L=3$ of 64. “Read the middle layer” is a Pythia-ism; for any new model, a layer sweep is mandatory before defining ρ .

C. Binary and hexadecimal: native-basis case studies

Binary (Table 3). At $n = 1024$ with $\mathcal{T} = \{2, 4, 8, 16, 32, 64\}$, fixing the basis recovers +0.10 to +0.37 helix R^2 over the paper basis at the same window. The biggest gains are on Qwen and Gemma, whose tokenizers split numerals into smaller sub-tokens, making the per-bit count structure more visible after mean-pooling. The random-embedding gap is $|\Delta R^2| \leq 0.03$ on every model: binary’s $L=0$ helix is mechanical, like Babylonian’s. Pythia/Llama/GPT-J/OLMo’s moderately-low native-basis ρ of 0.62–0.77 reflects depth doing additional work on top of a mechanical $L=0$, not learned bit semantics; the corresponding CKA values are 0.55–0.71.

Hexadecimal (Table 4). At $n = 1024$ with $\mathcal{T} = \{16, 32, 64, 256\}$, ΔR^2 is +0.19 to +0.29. The native-basis ρ on Pythia/Llama/GPT-J/OLMo is 0.29–0.35—in the same regime as the original paper’s Pythia/Latin (0.34) and Llama/Latin (0.39). Hex is the *second-cleanest* depth-built case in the matrix on those four models. The random-embedding gap is positive on every model (small on Llama/OLMo, sizeable on Gemma), so unlike binary the $L=0$ hex helix is partly learned. Gemma is more protocol-dependent: its paper-default on hex is 0.34 and 0.43 (in the depth-amplified regime), but its native-basis ρ rises to 0.54 and 0.63.

Table 3: Binary, paper basis vs. binary-native, $n = 1024$.

model	paper R^2	native R^2	Δ	native ρ
Pythia-6.9B	0.11	0.24	+0.13	0.62
GPT-J-6B	0.11	0.25	+0.13	0.71
Llama-3.1-8B	0.19	0.29	+0.10	0.73
Gemma-4-E4B	0.30	0.63	+0.32	1.00
Gemma-4-31B	0.34	0.63	+0.29	1.00
OLMo-3-32B	0.16	0.27	+0.11	0.77
Qwen2.5-7B	0.27	0.64	+0.37	1.00
Qwen2.5-32B	0.28	0.65	+0.37	1.00

Table 4: Hexadecimal, paper basis vs. hex-native, $n = 1024$.

model	paper R^2	native R^2	Δ	native ρ
Pythia-6.9B	0.05	0.29	+0.25	0.29
GPT-J-6B	0.06	0.27	+0.21	0.35
Llama-3.1-8B	0.05	0.34	+0.29	0.30
Gemma-4-E4B	0.16	0.36	+0.20	0.63
Gemma-4-31B	0.19	0.38	+0.19	0.54
OLMo-3-32B	0.06	0.26	+0.20	0.35
Qwen2.5-7B	0.06	0.35	+0.29	0.50
Qwen2.5-32B	0.07	0.36	+0.29	0.46

D. Random-embedding control across all scripts

Table 5 reports $\Delta R_{L=0}^2 = R_{\text{learned}}^2 - R_{\text{random}}^2$ for the paper-default protocol (extended-basis Babylonian/binary/hex deltas are in App. C and §4.3). Latin is large and positive on every model; Babylonian is near zero on every model. Per-digit positional scripts (Arabic-Indic, Persian, Devanagari, Thai) sit between: small positive Δ on Pythia/Llama/GPT-J/OLMo, larger positive on Gemma/Qwen whose tokenizers split digits finer (so even random embeddings carry more structure once mean-pooled, raising the floor without eliminating the learned contribution).

Table 5: Learned-vs-random $\Delta R_{L=0}^2$ across all paper-default cells. Negative or near-zero Δ marks mechanically-produced L=0 geometry.

script	Py	GPT-J	Llama	Gem-E4B	Gem-31B	OLMo	Qw-7B	Qw-32B
latin	+0.10	+0.12	+0.15	+0.12	+0.12	+0.18	+0.11	+0.03
arabic	+0.01	+0.01	+0.06	+0.13	+0.11	+0.03	+0.05	+0.04
persian	+0.01	−0.01	+0.09	+0.13	+0.11	+0.03	+0.03	+0.01
devanagari	+0.01	+0.00	+0.06	+0.13	+0.09	+0.03	+0.03	+0.03
thai	+0.01	+0.01	+0.01	+0.05	+0.04	+0.03	+0.02	+0.03
chinese	+0.03	+0.02	+0.07	+0.16	+0.14	+0.04	+0.03	+0.03
binary	+0.01	+0.02	−0.02	+0.05	+0.05	−0.03	+0.03	−0.01
hexadecimal	+0.03	+0.04	+0.04	−0.01	−0.02	+0.05	−0.01	−0.01
greek	+0.02	+0.02	+0.02	+0.04	+0.03	+0.03	+0.02	+0.02
hebrew	+0.02	+0.03	+0.01	+0.03	+0.03	+0.03	+0.06	+0.06
roman	+0.04	+0.05	+0.04	+0.15	+0.11	+0.03	+0.05	+0.04
babylonian	+0.00	−0.01	+0.01	+0.01	+0.01	−0.01	−0.00	−0.01

E. (ρ, CKA) scatter and the four quadrants

We compute linear CKA between the helix-projected L=0 representation and the helix-projected peak-layer representation for every cell. Four quadrants emerge in the (ρ, CKA) plane.

Pass-through (high ρ , high CKA): Qwen2.5-32B/Latin ($\rho = 0.89$, $\text{CKA} = 0.91$), OLMo/Latin (0.63, 0.91), Pythia $\times$ Persian/Devanagari/Chinese ($\text{CKA} \geq 0.98$). **Depth refines** (low-to-moderate ρ , high CKA): Pythia/Latin (0.34, 0.79), Llama/Latin (0.39, 0.81), Roman on every non-Gemma model ($\text{CKA} = 0.77\text{--}0.90$). **Rebuild** (high ρ , low CKA): the entire Gemma family ($\text{CKA} = 0.21\text{--}0.65$ across most scripts), Babylonian on Pythia/Llama/OLMo ($\rho \approx 0.86$, $\text{CKA} = 0.25\text{--}0.53$), Qwen on Devanagari ($\text{CKA} \approx 0.24\text{--}0.30$). **Depth builds new** (low ρ , low CKA):

rare; Gemma’s Devanagari/Persian/Roman/Greek cells edge into it ($\rho \approx 0.44\text{--}0.63$, $\text{CKA} \approx 0.29\text{--}0.57$).

The pass-through / rebuild distinction is the key new fact CKA contributes. The simpler reading—“Gemma/Latin $\rho \approx 0.85$, so mostly tokenizer”—was wrong. CKA on the same cell is only 0.33; depth produces a similar-quality helix with a substantially different integer-to-integer geometry. Across nearly every script, Gemma rebuilds. ρ alone made it look like Gemma was doing nothing; it is doing a lot, just not in a way that changes the score.

F. Classification grid (full)

The mutually-exclusive criteria of §2 applied to all 96 paper-default cells produce: depth-built = 6 (Pythia/Latin, GPT-J/Latin, Llama/Latin, Gemma-31B/Devanagari, Gemma-E4B/Devanagari, Gemma-31B/Binary—the last is in the protocol-mismeasured set); depth-amplified = 4 (OLMo/Latin, Gemma-31B/Persian, Gemma-E4B/Persian, Gemma-E4B/Binary); inherited = 35; weak = 36 (basis fits little); ambiguous = 15. Excluding the three protocol-mismeasured columns (binary, hex, Babylonian), the depth-built count is 5 out of 64.

G. Reproducibility

All code is public at https://github.com/girish/addition_helix (final URL on publication). Each driver script declares its dependencies via PEP 723 inline metadata and is runnable directly via `uv run`; one top-level `run.sh` executes the per-cell sweep (`main.py`), the aggregator (`aggregate.py`), the random-embedding control (`embed_control.py`), and the representation-alignment pass (`subspace_align.py`). The companion blogpost site (`www/`) reads the aggregated JSON and renders all figures interactively.

Compute. The full 96-cell sweep plus the three auxiliary passes completes in 6–8 hours on a single Apple M5 Max with 128 GB unified memory (MPS); fallback CUDA/CPU paths are in `main.py:pick_device`. The 32B-class models (Gemma-4-31B, OLMo-3-32B, Qwen2.5-32B) run in bf16. Each combination is idempotent: re-running skips any cell whose `fig_layer_sweep.json` already exists.

Data. All inputs are integers; we render them via `format_number` in `helix_lib.py` (12 scripts) and feed them through each model’s own tokenizer. No external corpus is used.